Densidad kernel con ancho de banda variable y comparación no paramétrica de la composición química de vinos italianos por cultivar

Estadística No Paramétrica · Propuesta definitiva del proyecto final · Profesor Ivan Ladino-Vega · Universidad Santo Tomás

Parte 1. Identificación

Título: Densidad kernel con ancho de banda variable y comparación no paramétrica de la composición química de vinos italianos por cultivar. **Integrantes:** Isabel Leal, Valentina Reyes y Hayni Avila. **Idea seleccionada:** caracterización química de vinos de tres cultivares (base Wine, UCI). **Repositorio:** https://github.com/reyes-785/Propuesta-Estad-stica-no-Param-trica.

Parte 2. Preguntas de investigación

- **Principal (P1).** ¿Difiere sistemáticamente el nivel de prolina (variable respuesta) entre los vinos de los cultivares 1, 2 y 3 de la región italiana de la base Wine (contraste: tres cultivares, con 59, 71 y 48 vinos)?
- **Secundaria 1 (S1).** En los 178 vinos de la base, ¿estima mejor la densidad de la prolina un estimador kernel con ancho de banda variable (tipo Abramson) que uno con ancho fijo, medido por la log-densidad de validación cruzada dejando uno fuera (contraste: dos estimadores sobre los mismos vinos)?
- **Secundaria 2 (S2).** ¿Es el contenido de flavonoides (variable respuesta) de los vinos del cultivar 1 sistemáticamente mayor que el de los vinos del cultivar 3 (contraste: dos cultivares, con 59 y 48 vinos)?

Parte 3. Datos

- **Fuente:** UCI Machine Learning Repository, conjunto «Wine» (Aeberhard, Coomans y de Vel), https://archive.ics.uci.edu/dataset/109/wine; versión con nombres de variables en español (wine_uci.csv). **Fecha de descarga:** 19 de octubre de 2026. **Filtros aplicados:** ninguno; el notebook de descarga reproduce la lectura y el renombrado de columnas.
- **Unidad de análisis:** un vino (una muestra con 13 constituyentes químicos y su cultivar). **Tamaño tras la limpieza:** n = 178 (cultivar 1: 59; cultivar 2: 71; cultivar 3: 48), sin pérdidas, porque no se eliminó ningún registro.
- **Variables usadas:** prolina (razón), flavonoides (razón), cultivar (nominal) e id_vino (identificador único). Las medianas de prolina por cultivar son 1095, 495 y 627,5, y las de flavonoides 2,98, 2,03 y 0,685.
- **Problemas detectados.** *Faltantes:* ninguno en las 178 × 15 celdas; tampoco hay filas duplicadas. *Empates:* abundantes por la resolución de medición; la prolina repite 57 de sus 178 valores (11, 17 y 12 por cultivar) y los flavonoides 46 (9, 13 y 12), por lo que se usarán correcciones por empates y, cuando sea posible, valores p por permutación. *Atípicos* (regla 1,5·RIC): ninguno en prolina ni flavonoides en el conjunto completo; por cultivar aparecen 1 en prolina (cultivar 2) y 2 en flavonoides (uno en el cultivar 2 y otro en el 3). Se conservan, pues los procedimientos basados en rangos son resistentes. *Dependencias:* no hay series de tiempo ni mediciones repetidas, y se supone independencia entre vinos, aunque la fuente no documenta el muestreo. La prolina global es una mezcla de tres subpoblaciones (asimetría 0,76), lo que motiva el ancho variable. Las dispersiones difieren entre cultivares (Fligner–Killeen, $p < 0{,}001$ para ambas variables), por lo que la condición de «forma similar» no se da por cumplida y se vigilará en cada contraste.

Parte 4. Mapa de hipótesis y procedimientos

Pregunta	Diseño	Hipótesis	Prueba (familia)	Supuestos y cómo se verifican	Efecto
P1. ¿Difiere la prolina entre los tres cultivares?	3 grupos independientes (59, 71, 48)	H_0: $F_1 = F_2 = F_3$ (igual distribución de la prolina); H_1: al menos dos cultivares difieren	Kruskal–Wallis + Dunn con Holm (F4)	Independencia entre vinos (id únicos, sin medidas repetidas); forma similar (KDE por cultivar y Fligner–Killeen; si falla, se interpreta como dominancia estocástica); empates corregidos	$\varepsilon^2_H = H/(n-1)$; en Dunn, probabilidad de superioridad por par
S1. ¿El ancho variable estima mejor la densidad de la prolina que el fijo?	178 prolinas; dos estimadores comparados por vino con $d_i = \log f_{A,-i}(x_i) - \log f_{F,-i}(x_i)$	H_0: mediana de $d_i = 0$; H_1: mediana de $d_i > 0$ (gana el ancho variable)	KDE con h variable vs h fijo (F5) + Wilcoxon de rangos con signo sobre d_i (F2)	Simetría de d_i (histograma y gráfico de cuantiles); dependencia leve de la validación cruzada, que se reconoce y se contrasta con permutación de signos; h de ambos estimadores por máxima verosimilitud dejando uno fuera, con $\alpha = 1/2$ fija (igual número de parámetros ajustados)	Ganancia media en log-densidad por vino y correlación biserial de rangos
S2. ¿Los flavonoides del cultivar 1 superan a los del 3?	2 grupos independientes (59 y 48)	H_0: $F_1 = F_3$; H_1: $P(X_1 > X_3) > 1/2$ (dominancia estocástica del cultivar 1)	Mann–Whitney unilateral (F3)	Independencia; empates corregidos en la varianza de U; forma similar para leer el resultado como desplazamiento de ubicación (KDE y Fligner–Killeen; si falla, se lee como dominancia estocástica)	$A = U/(n_1 n_3)$ (probabilidad de superioridad) y correlación biserial de rangos $r = 2A - 1$

Verificación de cobertura: cuatro procedimientos (Kruskal–Wallis con Dunn, Wilcoxon de rangos con signo, Mann–Whitney y estimador kernel) de cuatro familias (F2, F3, F4 y F5), una por procedimiento, con lo que se cumple el mínimo exigido. Las comparaciones múltiples de P1 usan el ajuste de Holm.

Parte 5. Plan de trabajo

Fundamento matemático. Sea f una densidad con cuatro derivadas continuas y K un núcleo simétrico con segundo momento finito. Se define el estimador adaptativo (piloto f_0) $f_A(x) = (1/n)\ \Sigma_i\ (h\lambda_i)^{-1}\ K((x - X_i)/(h\lambda_i))$, con $\lambda_i = (f_0(X_i)/g)^{-1/2}$ y g la media geométrica de los $f_0(X_i)$. Resultado a demostrar: si $h \to 0$ y $nh \to \infty$, y el piloto es ideal ($f_0 = f$), el sesgo de $f_A(x)$ es $O(h^4)$, frente a $O(h^2)$ con h fijo, y la varianza sigue siendo de orden $(nh)^{-1}$. La demostración parte de la expansión de Taylor de f en torno a x con ancho local $h\lambda(x)$; el efecto de un piloto estimado se estudiará por simulación. Como resultado complementario se demostrará que, bajo H_0 y sin empates, $E(H) = k - 1$ en Kruskal–Wallis.

Programación. Se implementarán desde cero en Python (numpy): (i) el KDE de ancho fijo y el adaptativo con piloto gaussiano y h elegido por máxima verosimilitud dejando uno fuera; (ii) el estadístico de Kruskal–Wallis con corrección por empates y las comparaciones de Dunn con Holm. Se compararán con scipy.stats.gaussian_kde, density(x, bw = "SJ") de R, scipy.stats.kruskal y scikit_posthocs.posthoc_dunn. Se agregará un estudio de simulación del error cuadrático integrado con mezclas normales bimodales y asimétricas, para medir cuándo gana el ancho variable.

Cronograma (semanas contadas desde la aprobación de la propuesta; las fechas se ajustan a la entrega final):

Semana	Tarea	Responsable
1	Repositorio en GitHub con datos y notebook de descarga; exploración y verificación de supuestos	Valentina Reyes
1–2	Demostración del sesgo y la varianza del estimador adaptativo; demostración de E(H) = k − 1	Isabel Leal
2–3	KDE fijo y adaptativo desde cero; validación frente a scipy y R	Hayni Avila
3–4	Kruskal–Wallis + Dunn (P1) y Mann–Whitney (S2)	Valentina Reyes
4–5	Comparación por validación cruzada y Wilcoxon (S1); simulación del error cuadrático integrado	Hayni Avila e Isabel Leal
6	Informe final, figuras y revisión conjunta	Las tres integrantes

Anexo. Código del notebook del repositorio

Se reproduce el contenido de las celdas de Propuesta_Estadistica_No_Parametrica.ipynb (descarga, análisis y gráficos), disponible en el repositorio del proyecto.

```
# ---- 0. Descarga y preparación de los datos ----

%pip install -q ucimlrepo

import datetime
from ucimlrepo import fetch_ucirepo

wine = fetch_ucirepo(id=109)   # Wine, UCI Machine Learning
    Repository
X, y = wine.data.features.copy(), wine.data.targets

X.columns = ["alcohol", "ac_malico", "ceniza", "alcal_ceniza",
    "magnesio",
             "fenoles_tot", "flavonoides", "fenoles_noflav",
                 "proantocianinas",
             "int_color", "tono", "od280_od315", "prolina"]
X.insert(0, "id_vino", [f"Vino_{i:03d}" for i in range(1, len(X)
    + 1)])
X["cultivar"] = "Cultivar " + y.iloc[:, 0].astype(str)

X.to_csv("wine_uci.csv", index=False)
print("Descarga ejecutada el", datetime.date.today())
print(X.shape)                                      # esperado:
    (178, 15)
print(X["cultivar"].value_counts().sort_index())  # esperado: 59,
     71, 48

# ---- 1. Librerías y lectura ----

%matplotlib inline
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt

rng = np.random.default_rng(2026)
df = pd.read_csv("wine_uci.csv")   # generado por la celda de
    descarga
cultivares = sorted(df["cultivar"].unique())
print("n =", len(df),
    df["cultivar"].value_counts().sort_index().to_dict())

# ---- 2. Utilidades (Holm) ----

# ============================================================
# Utilidades: Holm
# ============================================================
def holm(p):
    p = np.asarray(p, float)
    m = len(p)
    orden = np.argsort(p)
    ajust = np.empty(m)
    corrido = 0.0
    for k, i in enumerate(orden):
        corrido = max(corrido, (m - k) * p[i])
        ajust[i] = min(1.0, corrido)
    return ajust

# ---- 3. P1 · Kruskal–Wallis + Dunn con Holm (prolina por
    cultivar) ----

# ============================================================
# P1. Kruskal-Wallis con corrección por empates y Dunn con Holm
    (desde cero)
# ============================================================
def kruskal_wallis(grupos):
    todos = np.concatenate(grupos)
    N = len(todos)
    r = stats.rankdata(todos)
    lim = np.cumsum([0] + [len(g) for g in grupos])
    rangos = [r[lim[k]:lim[k + 1]] for k in range(len(grupos))]
    H = 12 / (N * (N + 1)) * sum(len(rk) * rk.mean() ** 2 for rk
        in rangos) - 3 * (N + 1)
    _, t = np.unique(todos, return_counts=True)
    C = 1 - (t ** 3 - t).sum() / (N ** 3 - N)          #
        corrección por empates
    H = H / C
    p = stats.chi2.sf(H, len(grupos) - 1)
    eps2 = H / (N - 1)                                    # tamaño
         del efecto
    return H, p, eps2, rangos, t, N

def dunn_holm(grupos, nombres):
    H, p, eps2, rangos, t, N = kruskal_wallis(grupos)
    sigma2 = N * (N + 1) / 12 - (t ** 3 - t).sum() / (12 * (N -
        1))   # con empates
    filas = []
    for i in range(len(grupos)):
        for j in range(i + 1, len(grupos)):
            ni, nj = len(grupos[i]), len(grupos[j])
            z = (rangos[i].mean() - rangos[j].mean()) /
                np.sqrt(sigma2 * (1 / ni + 1 / nj))
            pb = 2 * stats.norm.sf(abs(z))
            A = stats.mannwhitneyu(grupos[i],
                grupos[j]).statistic / (ni * nj)  # P(X_i > X_j)
                + 0.5 P(=)
            filas.append((nombres[i], nombres[j], z, pb, A))
    res = pd.DataFrame(filas, columns=["grupo_i", "grupo_j", "z",
         "p_bruto", "prob_superioridad"])
    res["p_Holm"] = holm(res["p_bruto"].values)
    return res

print("\n=== P1: prolina por cultivar ===")
g_prol = [df.loc[df.cultivar == c, "prolina"].values for c in
```

```
    cultivares]
H, p, eps2, *_ = kruskal_wallis(g_prol)
print(f"Kruskal-Wallis (desde cero): H = {H:.3f}, p = {p:.3g},
    eps2_H = {eps2:.3f}")
sp = stats.kruskal(*g_prol)
print(f"scipy.stats.kruskal:         H = {sp.statistic:.3f}, p =
    {sp.pvalue:.3g}")
print(dunn_holm(g_prol,
    cultivares).round(4).to_string(index=False))
try:
    import scikit_posthocs as sp_
    print("scikit_posthocs.posthoc_dunn (Holm):")
    print(sp_.posthoc_dunn(df, val_col="prolina",
        group_col="cultivar", p_adjust="holm").round(4))
except ImportError:
    print("(opcional) pip install scikit-posthocs para comparar
        con posthoc_dunn")

print("Fligner-Killeen (dispersiones iguales):",
    stats.fligner(*g_prol))

# Verificación del fundamento: bajo H0 y sin empates, E(H) = k -
    1
def simula_EH(tamanos, B=5000):
    Hs = np.empty(B)
    for b in range(B):
        Hs[b] = kruskal_wallis([rng.normal(size=n) for n in
            tamanos])[0]
    return Hs.mean()
print(f"Simulación E(H) con k=3: {simula_EH([len(g) for g in
    g_prol]):.3f} (teórico = 2)")

# ---- 4. S1 · Densidad de la prolina: h fijo vs h variable ----

# ============================================================
# S1. KDE gaussiano con h fijo y h variable (Abramson), desde
    cero
# ============================================================
def kde_fijo(x_eval, datos, h):
    z = (np.asarray(x_eval)[:, None] - datos[None, :]) / h
    return np.exp(-0.5 * z ** 2).sum(1) / (len(datos) * h *
        np.sqrt(2 * np.pi))

def lambdas_abramson(datos, h0, alpha=0.5):
    piloto = np.maximum(kde_fijo(datos, datos, h0), 1e-300)
    g = np.exp(np.mean(np.log(piloto)))                  # media
        geométrica
    return (piloto / g) ** (-alpha)

def kde_variable(x_eval, datos, h, h0, alpha=0.5):
    hi = h * lambdas_abramson(datos, h0, alpha)
    z = (np.asarray(x_eval)[:, None] - datos[None, :]) / hi[None,
         :]
    return (np.exp(-0.5 * z ** 2) / (hi[None, :] * np.sqrt(2 *
        np.pi))).mean(1)

def silverman(x):
    return 0.9 * min(x.std(ddof=1), stats.iqr(x) / 1.34) * len(x)
         ** (-0.2)

def loo_fijo(datos, hs):
    n = len(datos)
    ll = np.empty((len(hs), n))
    for i in range(n):
        d = np.delete(datos, i)
        zi = datos[i] - d
        for a, h in enumerate(hs):
            f = np.mean(np.exp(-0.5 * (zi / h) ** 2) / (h *
                np.sqrt(2 * np.pi)))
            ll[a, i] = np.log(max(f, 1e-300))
    return ll

def loo_variable(datos, hs, h0, alpha=0.5):
    n = len(datos)
    ll = np.empty((len(hs), n))
    for i in range(n):
        d = np.delete(datos, i)                          # el
            piloto tampoco usa x_i
        lam = lambdas_abramson(d, h0, alpha)
        zi = datos[i] - d
        for a, h in enumerate(hs):
            hi = h * lam
            f = np.mean(np.exp(-0.5 * (zi / hi) ** 2) / (hi *
                np.sqrt(2 * np.pi)))
            ll[a, i] = np.log(max(f, 1e-300))
    return ll

print("\n=== S1: densidad de la prolina, h fijo vs h variable
    ===")
x = df["prolina"].values.astype(float)
h0 = silverman(x)
hs = h0 * np.geomspace(0.2, 3.0, 40)

# Validación contra scipy (h fijo): gaussian_kde usa h = factor *
     sd
rejilla = np.linspace(x.min() - 150, x.max() + 150, 400)
h_prueba = h0
f_mio = kde_fijo(rejilla, x, h_prueba)
f_scipy = stats.gaussian_kde(x, bw_method=h_prueba /
    x.std(ddof=1))(rejilla)
print("KDE fijo propio == scipy.gaussian_kde:",
    np.allclose(f_mio, f_scipy))
# En R: density(x, bw = h_prueba, kernel = "gaussian") debe dar
    lo mismo;
#       density(x, bw = "SJ") para el ancho de Sheather-Jones.

ll_f = loo_fijo(x, hs)
ll_v = loo_variable(x, hs, h0)
a_f, a_v = ll_f.mean(1).argmax(), ll_v.mean(1).argmax()
h_f, h_v = hs[a_f], hs[a_v]
print(f"h0 (Silverman) = {h0:.1f} | h fijo óptimo = {h_f:.1f} | h
     variable óptimo = {h_v:.1f}")
print(f"log-verosimilitud LOO media: fijo =
    {ll_f[a_f].mean():.4f}, variable = {ll_v[a_v].mean():.4f}")

d = ll_v[a_v] - ll_f[a_f]                                # d_i
    por vino
print(f"Ganancia media por vino (variable - fijo) =
    {d.mean():.4f}")
w = stats.wilcoxon(d, alternative="greater")
dnz = d[d != 0]
rangos = stats.rankdata(np.abs(dnz))
r_rb = (rangos[dnz > 0].sum() - rangos[dnz < 0].sum()) /
    rangos.sum()
print(f"Wilcoxon (H1: mediana d > 0): W = {w.statistic:.1f}, p =
    {w.pvalue:.4g}, r_rb = {r_rb:.3f}")

# Permutación de signos sobre la media de d_i (la validación
    cruzada induce dependencia leve)
B = 20000
signos = rng.choice([-1, 1], size=(B, len(d)))
perm = (signos * d).mean(1)
p_perm = (np.sum(perm >= d.mean()) + 1) / (B + 1)
print(f"Permutación de signos (media de d_i): p = {p_perm:.4g}")

# ---- Gráfico: histograma y KDE (h fijo vs h variable) ----

# Gráfico: histograma + KDE fijo + KDE variable
fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(x, bins=25, density=True, color="#DDE3EA",
    edgecolor="white")
ax.plot(rejilla, kde_fijo(rejilla, x, h_f), label=f"h fijo =
    {h_f:.0f}")
ax.plot(rejilla, kde_variable(rejilla, x, h_v, h0), label=f"h
    variable (Abramson), h = {h_v:.0f}")
ax.set_xlabel("Prolina"); ax.set_ylabel("Densidad"); ax.legend()
fig.tight_layout(); fig.savefig("densidad_prolina.png", dpi=150);
     plt.show()

# ---- Gráfico: KDE por cultivar (condición de forma similar de
    P1) ----

# KDE por cultivar (para revisar la condición de forma similar de
     P1)
fig, ax = plt.subplots(figsize=(7, 4))
for c, g in zip(cultivares, g_prol):
    hc = silverman(g.astype(float))
    ax.plot(rejilla, kde_fijo(rejilla, g.astype(float), hc),
        label=c)
ax.set_xlabel("Prolina"); ax.set_ylabel("Densidad"); ax.legend()
fig.tight_layout(); fig.savefig("kde_prolina_por_cultivar.png",
    dpi=150); plt.show()

# ---- 5. S2 · Mann–Whitney unilateral (flavonoides, cultivar 1
    vs 3) ----

# ============================================================
# S2. Mann-Whitney unilateral: flavonoides, cultivar 1 vs
    cultivar 3
# ============================================================
print("\n=== S2: flavonoides, Cultivar 1 vs Cultivar 3 ===")
a = df.loc[df.cultivar == "Cultivar 1", "flavonoides"].values
b = df.loc[df.cultivar == "Cultivar 3", "flavonoides"].values
mw = stats.mannwhitneyu(a, b, alternative="greater")      # H1:
    P(X1 > X3) > 1/2
A = mw.statistic / (len(a) * len(b))
print(f"U = {mw.statistic:.1f}, p = {mw.pvalue:.3g}")
print(f"Probabilidad de superioridad A = {A:.3f}; correlación
    biserial de rangos r = {2 * A - 1:.3f}")
print("Fligner-Killeen (cultivar 1 vs 3):", stats.fligner(a, b))
```